

```

        if (count[k] > 0) for (m=0;m<mm;m++) means[k][m] /= count[k];
    }
}
};

```

Incidentally, k-means clustering is not only a simplification of Gaussian mixture models; it is actually a limiting case. If the Σ_k matrices are all held fixed as

$$\Sigma_k = \epsilon \mathbf{1} \quad (16.1.16)$$

with ϵ infinitesimal and $\mathbf{1}$ the identity matrix, then the component k with mean closest to $\mathbf{x}_n$ will be assigned all of the responsibility p_{nk} for that n . The re-estimation of the μ_k 's then is identical to k-means clustering. The theorem that proves that the EM algorithm converges for Gaussian mixtures can easily be modified to prove the convergence of k-means clustering. (Basically, there is a hidden log-likelihood function that can be shown to increase at each step.)

Indeed, we can now explain the obscure constant $1.0\text{e-}10$ in the initialization part of Gaumixmod: It is a value for ϵ that makes that routine's *first* E-step, M-step iteration be one of k-means clustering.

CITED REFERENCES AND FURTHER READING:

- McLachlan, G. and Peel, D. 2000, *Finite Mixture Models* (New York: Wiley).
- Moore, A.W. 2004, "Clustering with Gaussian Mixtures," at <http://www.cs.cmu.edu/~awm>.
- Dempster, A.P., Laird, N.M., and Rubin, D.B. 1977, "Maximum Likelihood from Incomplete Data via the EM Algorithm," *Journal of the Royal Statistical Society, Series B*, vol. 39, pp. 1-38. [The original paper on EM methods.]
- Tanner, M.A. 2005, *Tools for Statistical Inference: Methods for the Exploration of Posterior Distributions and Likelihood Functions*, 3rd ed. (New York: Springer).

16.2 Viterbi Decoding

In this section we discuss models with discrete states, and how to use data to estimate what state a model is in, or what succession of states it traverses by allowed transitions. By *state*, we mean some discrete condition that can be characterized as a node on a directed graph like that in Figure 16.2.1. By *transition*, we mean moving along one of the directed edges of the graph. If you want to characterize a continuous variable in the context of this section, you need to define a set of discrete bins for its possible values, and make these the states.

The setup we describe is slightly more general than its close cousin, the directed graph of *stages and states* that defined the dynamic programming (DP) problem in §10.13. For some applications, the estimation problem of interest does live on a graph that has states and stages, exactly like DP; but for other applications, we need a general directed graph. We'll consider both types below.

Historically, problems involving the estimation of states have arisen in diverse, and often noncommunicating, fields. There are often multiple names for single concepts. (We saw this previously in the Bellman-Dijkstra-Viterbi algorithm for DP.) This history also makes it hard to give, in this section, a unified treatment with a single narrative. A more practical approach is to go through a couple of examples from different fields, and then, afterward, make some comparisons and give some advice.

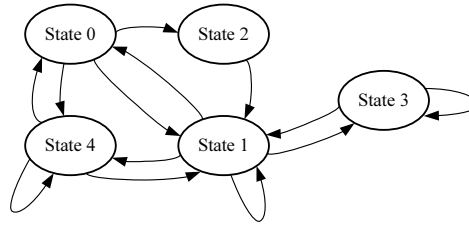


Figure 16.2.1. Graph of a dynamical system with five states. Allowed transitions are shown by arrows.

16.2.1 Error-Correcting Codes and Soft-Decision Decoding

An (N, K) binary block code is a list of 2^K binary codewords, each of length $N > K$ bits, designed to send a K -bit message in such a way that it can be received correctly even if one or more of the N bits arrive garbled (that is, 0 instead of 1, or vice versa). Two simple examples are shown below. In these particular cases, the message bits are the initial bits of the codeword, but that need not be true in general. Assigning any permutation of the codewords to message words is, effectively, the same code; likewise an arbitrary permutation of the bits in all the codewords (permuting bit-columns in the table).

(6,3) Shortened Hamming	
message	codeword
000	000000
001	001110
010	010101
011	011011
100	100011
101	101101
110	110110
111	111000

(7,4) Hamming	
message	codeword
0000	0000000
0001	0001011
0010	0010111
0011	0011100
0100	0100110
0101	0101101
0110	0110001
0111	0111010
1000	1000101
1001	1001110
1010	1010010
1011	1011001
1100	1100011
1101	1101000
1110	1110100
1111	1111111

Both of the codes shown have the property that their *Hamming distance* is 3. This means that all pairs of codewords differ in at least three bits. This is the property of the code that makes it “error correcting on one bit.” If you receive a codeword with one of its bits wrong, then (i) it will not be in the above table, and (ii) there will be a unique codeword in the table that differs from it in one bit position. So, trying each bit position in turn, you can figure out what was the intended codeword.

A longer code, with a larger Hamming distance d , can be error correcting for more than one garbled bit, in fact for $(d - 1)/2$ bits (rounding down). An (N, K) code can have d as large as $N - K$. However, trying all possible corrections until you find a valid codeword is a very poor decode strategy!

For so-called *linear codes* it is possible to construct a *parity-check matrix* $\mathbf{P}$ with the property that multiplying it by the vector of received bits (and doing all

arithmetic modulo 2) gives a vector, the so-called *syndrome*, that is either all zeros (indicating that the received bits are ok) or else it uniquely corresponds to a mask (termed a *coset leader*) that tells which bits need correcting. So this error-correction algorithm, called *syndrome decoding*, can be summarized as:

- multiply the received bits by the parity-check matrix to get the syndrome,
- do a table lookup of the syndrome to get the coset leader, and
- XOR the coset leader with the received bits to get a valid codeword.

For example, the parity check matrix for the (7, 4) Hamming code, above, is

$$\mathbf{P} = \begin{pmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{pmatrix} \quad (16.2.1)$$

(Convert codewords to column vectors by reading from left to right.) The lookup table relating syndrome to coset leader is

syndrome	coset leader
000	0000000
001	0000001
010	0000010
011	0001000
100	0000100
101	1000000
110	0100000
111	0010000

This particular code is called a *perfect code* because the number of syndromes exactly equals the number of coset leaders with one nonzero bit, plus 1 for the zero syndrome, a numerological coincidence. There are very few perfect codes, because there are very few sets of integers satisfying

$$1 + \binom{N}{1} + \cdots + \binom{N}{e} = 2^{N-K} \quad (16.2.2)$$

where e is the number of bits corrected. Probably the most nontrivial perfect code is the *Golay code*, with $N = 23$, $K = 12$, and $e = 3$. (Check out the numerology yourself.)

It's no big deal if a code is not perfect. It just means that there are some extra syndromes that correct *some* errors of more than e bits, but not enough to correct all such errors. You include these extra syndromes in the table, and run the algorithm exactly as already described. However, if a code is too far from perfect, you are wasting syndromes without gaining more bits of sure correction.

In practical applications, N and K are larger than these examples. For example, the lowest level of error correction on an audio compact disk (CD) is a (28, 24) *Reed-Solomon* (RS) code, which can correct $e = 2$ bits. (On a CD, bits of the output codewords from many consecutive blocks are then interleaved and further protected by an RS(32,28) code.) Reed-Solomon codes are typically decoded by a more efficient process than syndrome decoding, using the so-called *Berlekamp-Massey* algorithm.

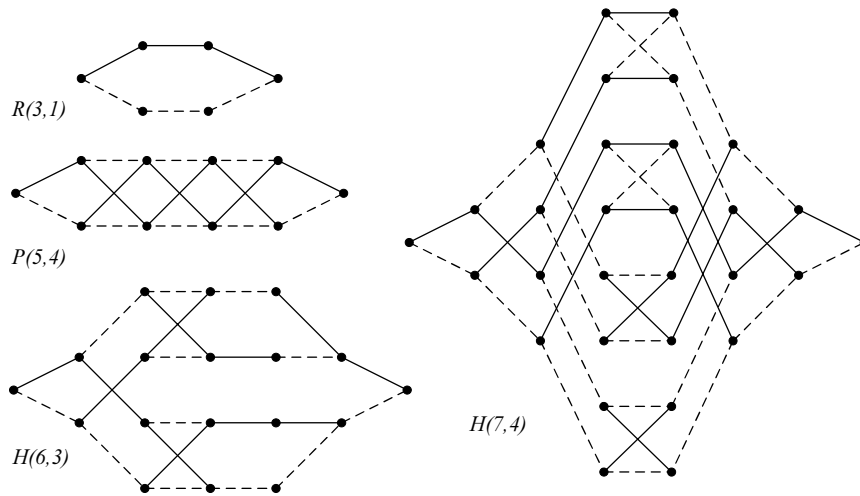


Figure 16.2.2. Trellises associated with four binary codes. The graph is traversed from left to right. A zero is output when any dotted edge is traversed, a one for a solid edge. Every path yields a valid codeword.

Now take a deep breath. Everything we have discussed thus far is what is called *hard-decision decoding* (HDD), meaning that hard decisions are made as to whether each incoming bit is a 1 or 0, with the error-correction algorithm acting on the resulting, possibly garbled, codeword. Virtually all coding theory utilized HDD until the early 1970s. Then came the giant leap forward with the recognition by multiple practitioners that Viterbi’s 1967 decoding algorithm (an independent rediscovery of Bellman-Dijkstra, we might now say) could utilize “soft” data about each bit as easily as hard.

To understand *soft-decision decoding* (SDD), let us first note that every binary code can be represented by the kind of stage/state graph that we met in dynamic programming (§10.13), which, in the present context, is called a *trellis*. Figure 16.2.2 shows the trellises for the two codes given explicitly above, as well as for the school-book examples of a repetition code (“tell me three times”) and a parity code. The latter, with $d = 1$, is error detecting, but not error correcting.

Although arrows are not shown, the trellis is traversed from left to right. Any such path on the trellis generates a valid codeword. A zero bit is emitted when a dotted edge is traversed, a one bit for a solid edge. You encode message bits by deciding whether to branch up or down, when you have a choice. Notice that you don’t get such a choice at every stage: “Forced” edges generate precisely the extra codeword bits that the code’s redundancy requires.

Although every code has a trellis, it is not so easy to find the *minimal trellis*, the one that has the fewest possible states at its maximum expansion. MacKay [1] gives a brief introduction; many additional references are in [3].

The first great idea behind soft-decision decoding is that we don’t need to decide whether an incoming bit is a 0 or 1. Rather, we just need to assign a probability to each possibility (summing to unity, of course). For example, a bit’s value may be determined by whether an instantaneous voltage is positive or negative — but the voltage measurement has some Gaussian spread of errors. If the voltage is many standard deviations positive, or negative, then the respective probabilities are very

close to one or zero; but if the voltage is only $t = 0.5$ (say) standard deviations away from zero, we may want to assign a probability of 0.6915 to one more favored outcome, and 0.3085 to the less favored, since

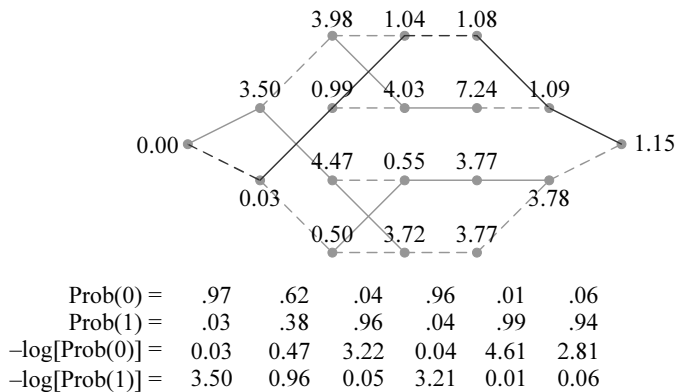
$$\frac{1}{\sqrt{2\pi}} \int_{-\infty}^{0.5} e^{-z^2/2} dz \approx 0.6915 \quad (16.2.3)$$

(By the end of this section we will be more sophisticated about the notion of assigning probabilities to transitions.)

The second great idea is that the problem of finding the maximum likelihood path through a trellis — that is, the path with the maximum product of the probabilities at each stage — is just a dynamic programming problem, where the cost of traversing an edge whose probability is p is taken as $-\log(p)$, a positive number, since $0 \leq p \leq 1$. The minimum cost path, with this metric, is the maximum likelihood path. In each stage, all the 0 edges (dotted lines in the figure) get one probability, and all the 1 edges (solid lines) get its complement. The edge probabilities can, and in general will, vary with each bit received (that is, from stage to stage), since the noise and path loss can vary with time.

Take these two ideas together and you have *soft-decision decoding using the Viterbi algorithm*.

The following tableau decodes one codeword for the shortened Hamming (6, 3) code given above. In this example, five of the six bits are received fairly unambiguously, while one bit (the second) is seen to be rather ambiguous. Nevertheless, the algorithm treats all bits equally “softly.” Reviewing §10.13 as necessary, you should be able to see where all the numbers in the tableau come from, as well as how the darker path (which is the final “hard” decision for the codeword 011011) is obtained by backtracking. Given the appropriate cost function, the routine `dynpro` in §10.13 does exactly this calculation.



You might have the “bright idea” of converting the final minimum path length, 1.15 in the above example, into a probability by taking its negative exponential. The result is 0.3166. Does this mean that you’ll get the right codeword only 31% of the time? No! Go stand in the corner! You are confusing likelihood with probability. The likelihoods of all eight codewords (not found by the DP algorithm, but computed exhaustively) are, for this example,

codeword	likelihood
000000	0.000014
001110	0.001372
010101	0.000006
011011	0.316126
100011	0.000665
101101	0.000007
110110	0.000001
111000	0.000006

You can see that the likelihoods don't add up to 1, and that the favored path wins over any competitor by a large factor. (Bayesians: We know that this paragraph is making you break out in a rash. We are on your side, and will have more to say about this in §16.3.4.)

In the above example, the second bit was merely ambiguous. What if it had instead been really wrong, indicating, say, 0.99 probability of having a value zero? No problem. Since the underlying code is one-bit error correcting, the DP algorithm will readily decide to traverse the unlikely single edge, since the alternative would be to traverse two or more unlikely edges on other bits. However, if we made the second bit incorrect with probability 0.999999, the algorithm would “correct” two other bits instead, which, under the circumstances, would be the best decision.

You can see that it is not meaningful to say exactly how many bits e a soft-decision decode algorithm can correct. It just makes the best choice determined by the probabilities. As another example, we might consider the simple parity code shown in Figure 16.2.2. With a hard-decision decode, parity does not give enough information to correct a single bit. With a soft-decision algorithm, however, the parity bit can cast the deciding vote if some other bit is wavering too close to an ambiguous 50% probability level.

Soft-decision decoding algorithms are available for essentially all codes in use today, including Reed-Solomon codes and the important *turbo codes* [2] that are beyond our scope. Some important applications (e.g., *trellis coded modulation*), use short trellises whose end states loop around to become identified with their start states. The Viterbi algorithm is applied to long sequences of input symbols that loop through the trellis many times. In trellis coded modulation, the symbols being (softly) decoded are not single bits, but locations in the complex phase plane that comprise a carefully chosen *constellation* centered at the origin (for example, a hexagonal lattice).

CITED REFERENCES AND FURTHER READING:

- Lin, S. and Costello, D.J. 2004, *Error Control Coding*, 2nd ed. (Upper Saddle River, NJ: Pearson-Prentice Hall).
- Blahut, R.E. 2002, *Algebraic Codes for Data Transmission* (Cambridge, UK: Cambridge University Press).
- MacKay, D.J.C. 2003, *Information Theory, Inference, and Learning Algorithms* (Cambridge, UK: Cambridge University Press).[1]
- Schlegel, C. and Perez, L. 2000, *Trellis and Turbo Coding*, (Piscataway, NJ: IEEE Press).[2]
- “Special Issue on Codes and Complexity”, 1996, *IEEE Transactions on Information Theory*, vol. 42, no. 6, pp. 1649-2064.[3]